DS Assignment 2 Report

Daniel Schlesinger SCHDAN066

22/03/2024

Page 1: Cover

Page 2: OO design

Page 2: Experimentation

Page 2: Test query values

Page 3: Instrumentation experimentation results

Page 4: Creativity

Page 4: Git log

OO design

My project consists of one main class called GenericsKbAVLApp that handles the execution of the program and contains the randomisation function used in the experimentation process. This class also has a static inner class called AVL which is the implementation of our data structure (AVL Tree) and handles the construction and methods that are related to our data structure such as insert, search, balance, etc. AVL is static so that it doesn't need an instance of our main class to be instantiated. AVL itself has an inner class called Node which is what we use to store all the different data entries in our dataset.

Experimentation

The goal of our experiment was to calculate the efficiency of our search and insert functions inside of an AVL Tree by looking at their time complexity. We did this by adding counters inside of our methods that would keep track of every comparison that takes place. By doing this, we can vary the dataset size and retrieve outputs from our programs that tell us how many comparisons take place and by extension, what the time complexity and efficiency of our AVL tree operations are.

Test Query Values

Input File → System Output:

population → population Populations are used for consumption. 1.0

fitting room → Term not found: fitting room

criminal organisation → Term not found: criminal organization

weft → weft A weft is thread 1.0

cilium → cilium A cilium is an organ 1.0

urban sprawl → Term not found: urban sprawl

meal moth → meal moth Meal moths have tints. 1.0

alkaline battery → Term not found: alkaline battery

fine silica → fine silica Fine silica affects characteristics. 1.0

organic gardening → Term not found: organic gardening

Instrumentation Experimentation Results

Data size	Insert Comparisons	Data size	Search Comparisons
1	0	1	10000
5	8	5	26750
27	91	27	49212
50	222	50	57721
275	1886	275	83661
500	3856	500	91928
2750	27953	2750	115632
5000	55202	5000	124403
27500	373975	27500	143653
50000	724248	50000	146418

Figure 1

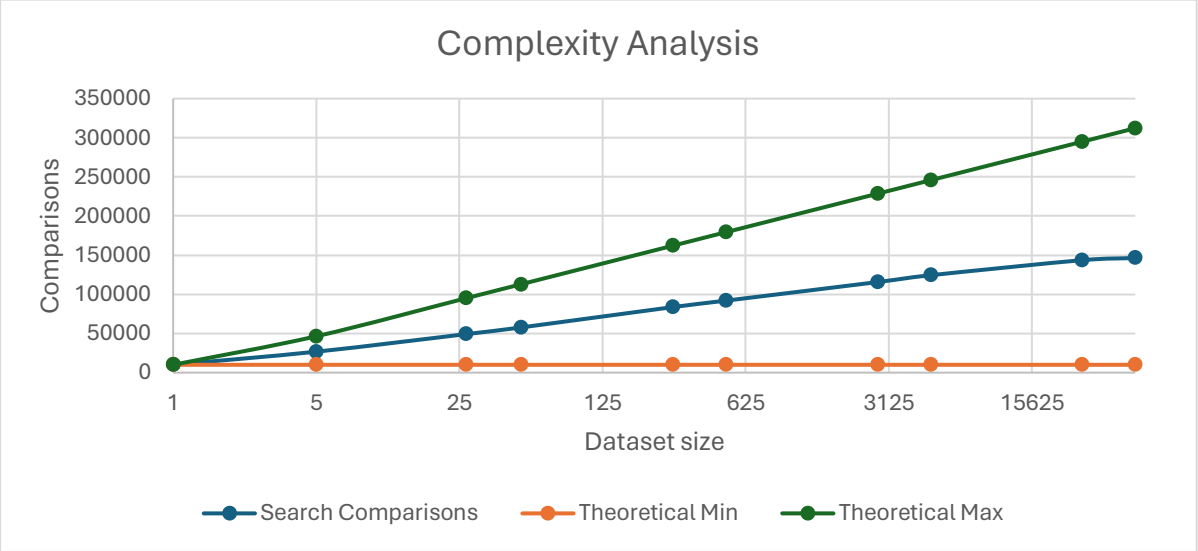


Figure 2

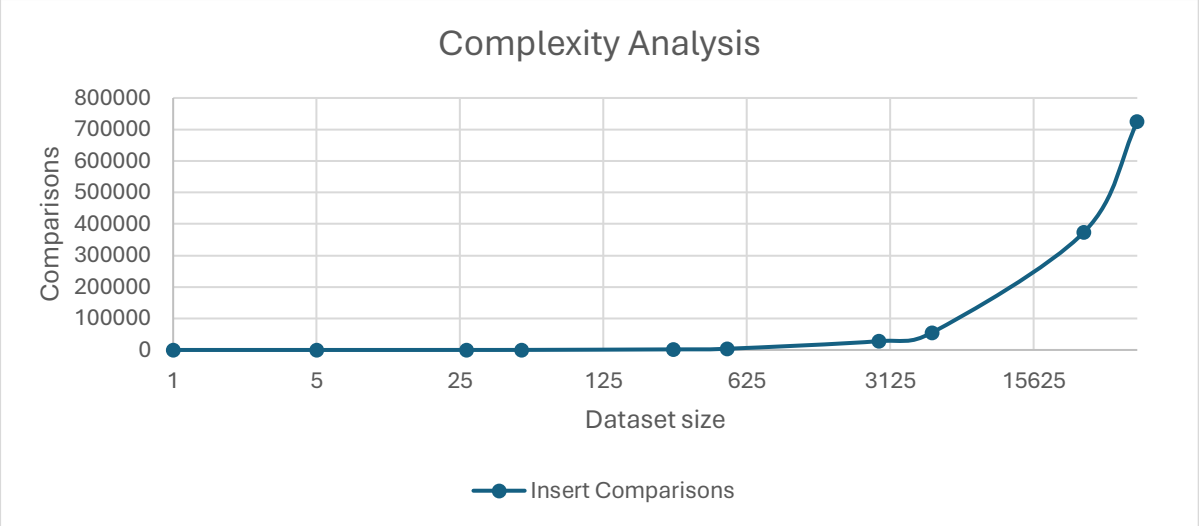


Figure 3

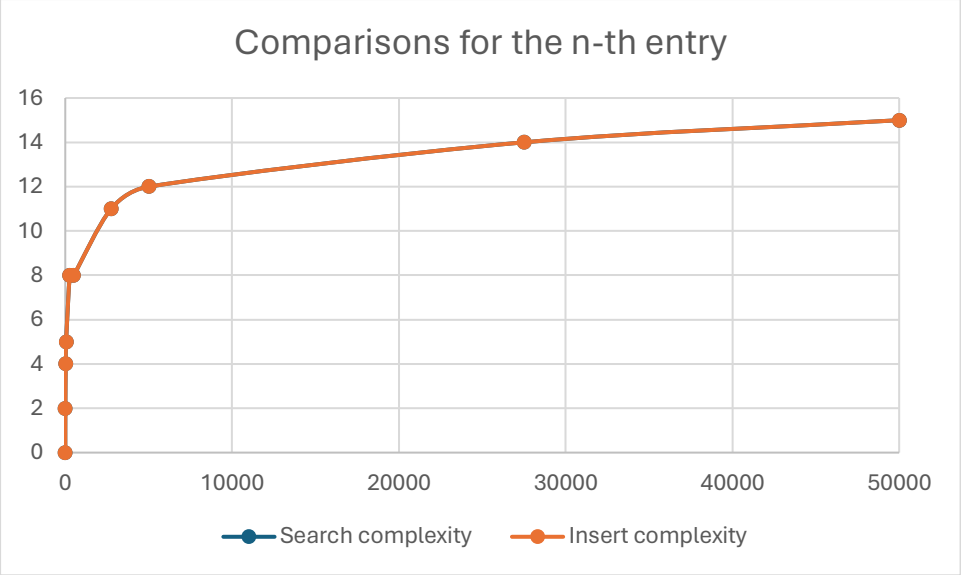


Figure 4

Analysis

As shown in figure 2, our search efficiency correlates to a graph of logN, in other words, it is O(logN). Since it is a balanced tree, the best, worst =, and average time efficiency for searching will all be logN graphs of varying constant multiples. This makes sense since the theoretical time complexity of an AVL Tree search is O(logN).

As shown in figure 3, our search efficiency also correlates to a logN graph. I did not manage to plot the best and worst cases, but they are also logN graphs. This lines up with our theoretical expectation as well since the time complexity of inserting into an AVL is O(logN).

Figure 4 shows the search and insert complexity for the nth data entry into the dataset. For individual entries the best case will be 0 comparisons and the worst case will be 15. The average amount of comparisons for insert and search is thus the same, coming out to 14.17 for the case of 50000 entries.

Creativity

I created a static randomIndices method that returned an array of a given size with the numbers from 0 to that size – 1 in a completely random order. I then used this method when I was creating subsets of GenericsKb.txt.

Git Log

```
commit 68c345240784459be7819ef4b3e7447160b11339
Author: Daniel Schlesinger <schdan066@myuct.ac.za>
Date:   Fri Mar 22 11:16:11 2024 +0200

    updated display and improved randomisation

commit 85fc3eb2b8d0bd5b7967d27697dd6b9d77d5d47f
Author: Daniel Schlesinger <schdan066@myuct.ac.za>
Date:   Fri Mar 22 10:55:26 2024 +0200

...
Author: Daniel Schlesinger <schdan066@myuct.ac.za>
Date:   Thu Mar 21 12:39:34 2024 +0200

    Finished part 1, still have to do output redirection but that is part of makefile

commit 83d8f7c4922aea572de0270ac531642be2de5093
Author: Daniel Schlesinger <schdan066@myuct.ac.za>
Date:   Thu Mar 21 10:58:39 2024 +0200

    Start of project
```